

```
[2]: import pandas as pd
import mysql.connector
import os

# List of CSV files and their corresponding table names
csv_files = [
    ('customers.csv', 'customers'),
    ('orders.csv', 'orders'),
    ('sellers.csv', 'sellers'),
    ('products.csv', 'products'),
    ('geolocation.csv', 'geolocation'),
    ('payments.csv', 'payments'),
    ('order_items.csv', 'order_items'), # Added payments.csv for specific handling
]

# Connect to the MySQL database
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='123456789',
    database='ecommerce',
)
cursor = conn.cursor()

# Folder containing the CSV files
folder_path = 'C:/Users/adyam/OneDrive/Desktop/Ecommerce'

def get_sql_type(dtype):
    if pd.api.types.is_integer_dtype(dtype):
        return 'INT'
    elif pd.api.types.is_float_dtype(dtype):
        return 'FLOAT'
```

```

elif pd.api.types.is_float_dtype(dtype):
    return 'FLOAT'
elif pd.api.types.is_bool_dtype(dtype):
    return 'BOOLEAN'
elif pd.api.types.is_datetime64_any_dtype(dtype):
    return 'DATETIME'
else:
    return 'TEXT'

for csv_file, table_name in csv_files:
    file_path = os.path.join(folder_path, csv_file)

    # Read the CSV file into a pandas DataFrame
    df = pd.read_csv(file_path)

    # Replace NaN with None to handle SQL NULL
    df = df.where(pd.notnull(df), None)

    # Debugging: Check for NaN values
    print(f"Processing {csv_file}")
    print(f"NaN values before replacement:\n{df.isnull().sum()}\n")

    # Clean column names
    df.columns = [col.replace(' ', '_').replace('-', '_').replace('.', '_') for col in df.columns]

    # Generate the CREATE TABLE statement with appropriate data types
    columns = ', '.join([f"`{col}` {get_sql_type(df[col].dtype)}" for col in df.columns])
    create_table_query = f'CREATE TABLE IF NOT EXISTS `{table_name}` ({columns})'
    cursor.execute(create_table_query)

    # Insert DataFrame data into the MySQL table
    for _, row in df.iterrows():
        # Convert row to tuple and handle NaN/None explicitly
        values = tuple(None if pd.isna(x) else x for x in row)

```

```

# Convert row to tuple and handle NaN/None explicitly
values = tuple(None if pd.isna(x) else x for x in row)
sql = f"INSERT INTO `{table_name}` ({', '.join(['`' + col + '`' for col in df.columns])}) VALUES ({', '.join(['%s' * len(row)])})"
cursor.execute(sql, values)

# Commit the transaction for the current CSV file
conn.commit()

# Close the connection
conn.close()

```

Processing customers.csv

NaN values before replacement:

customer_id	0
customer_unique_id	0
customer_zip_code_prefix	0
customer_city	0
customer_state	0

dtype: int64

Processing orders.csv

NaN values before replacement:

order_id	0
customer_id	0
order_status	0
order_purchase_timestamp	0
order_approved_at	160
order_delivered_carrier_date	1783
order_delivered_customer_date	2965
order_estimated_delivery_date	0

dtype: int64

Processing sellers.csv

NaN values before replacement:

seller_id	0
seller_zip_code_prefix	0

```
[4]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import mysql.connector
import numpy as np
db = mysql.connector.connect(host='localhost',
                             user='root',
                             password='854369aadi',
                             database='ecommerce',
                             )
cur = db.cursor()
```

List all unique cities where customers are located.

```
22]: query = """select distinct customer_city from customers """
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data)
df.head()
```

```
22]:
```

	0
0	franca
1	sao bernardo do campo
2	sao paulo
3	mogi das cruzeiras
4	campinas

Count the number of orders placed in 2017

[5]:

```
query = """select count(order_id) from orders where year(order_purchase_timestamp) = 2017"""
cur.execute(query)
data = cur.fetchall()
"total orders placed in 2017 are",data[0][0]
```

[5]: ('total orders placed in 2017 are', 45101)

Find the total sales per category

[11]:

```
query = """select upper(products.product_category) category,
round(sum(payments.payment_value),2)sales
from products join order_items
on products.product_id = order_items.product_id
join payments
on payments.order_id = order_items.order_id
group by category"""
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data,columns = ["Category","Sales"])
df
```

[11]:

	Category	Sales
0	PERFUMERY	506738.66
1	FURNITURE DECORATION	1430176.39
2	TELEPHONY	486882.05
3	BED TABLE BATH	1712553.67
4	AUTOMOTIVE	852294.33
...	...	...
69	CDS MUSIC DVDS	1199.43
70	LA CUISINE	2913.53
71	FASHION CHILDREN'S CLOTHING	785.67
72	PC GAMER	2174.43
73	INSURANCE AND SERVICES	324.51

74 rows × 2 columns

Calculate the percentage of orders that were paid in installments.

[15]:

```
query = """select (sum(case when payment_installments >= 1 then 1
else 0 end))/count(*)*100 from payments
"""
```

```
cur.execute(query)
```

```
cur.execute(query)
data = cur.fetchall()
"the percentage of orders that were paid in installments is ", data[0][0]

('the percentage of orders that were paid in installments is ',
 Decimal('99.9981'))
```

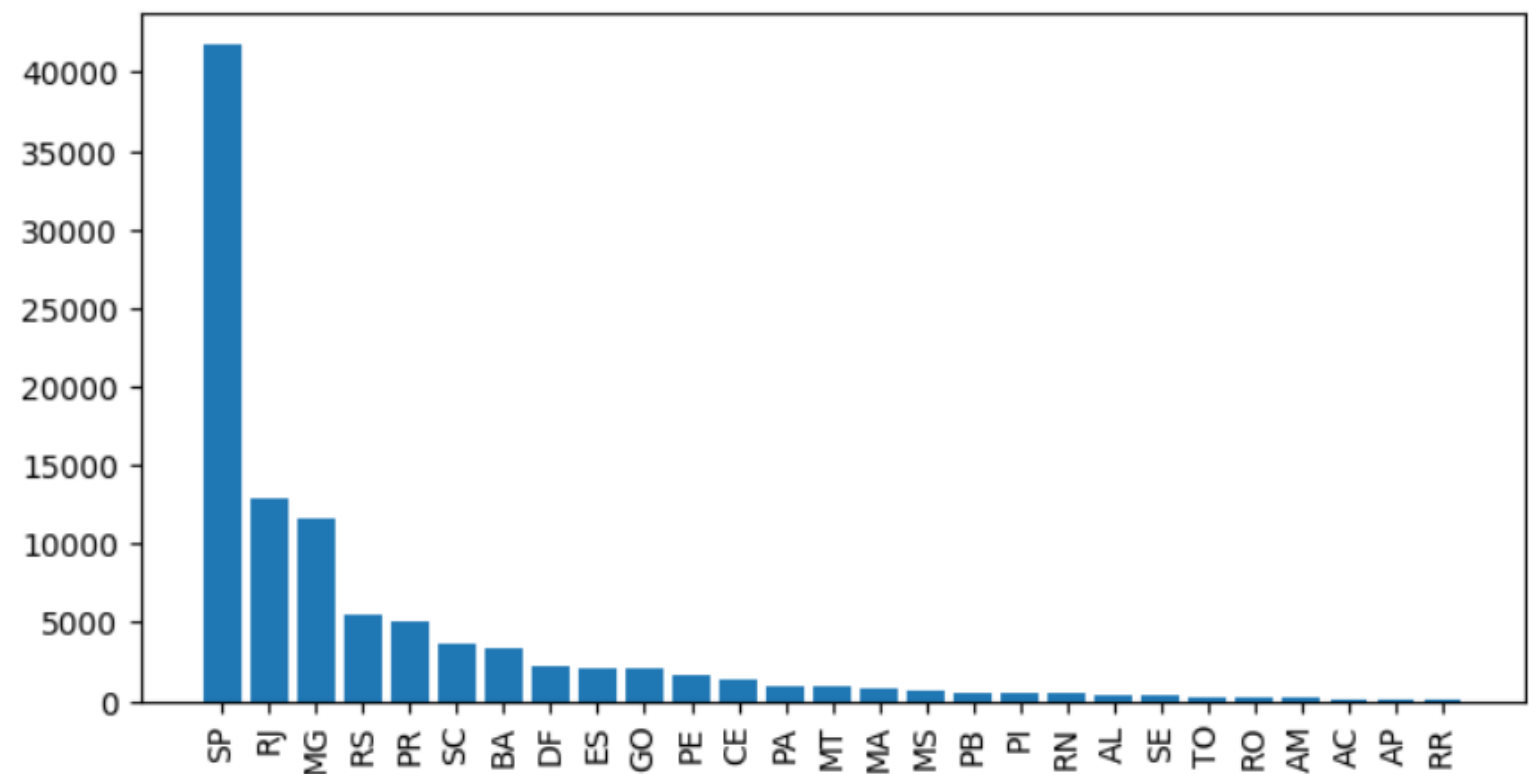
Count the number of customers from each state .

```
query = """select customer_state ,count(customer_id)
from customers group by customer_state
"""

cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data, columns = ["state","customer_count"])
df = df.sort_values(by = "customer_count",ascending = False)
plt.figure(figsize = (8,4))
plt.bar(df["state"],df["customer_count"])
plt.xticks(rotation = 90)
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```

[21]: <function matplotlib.pyplot.show(close=None, block=None)>



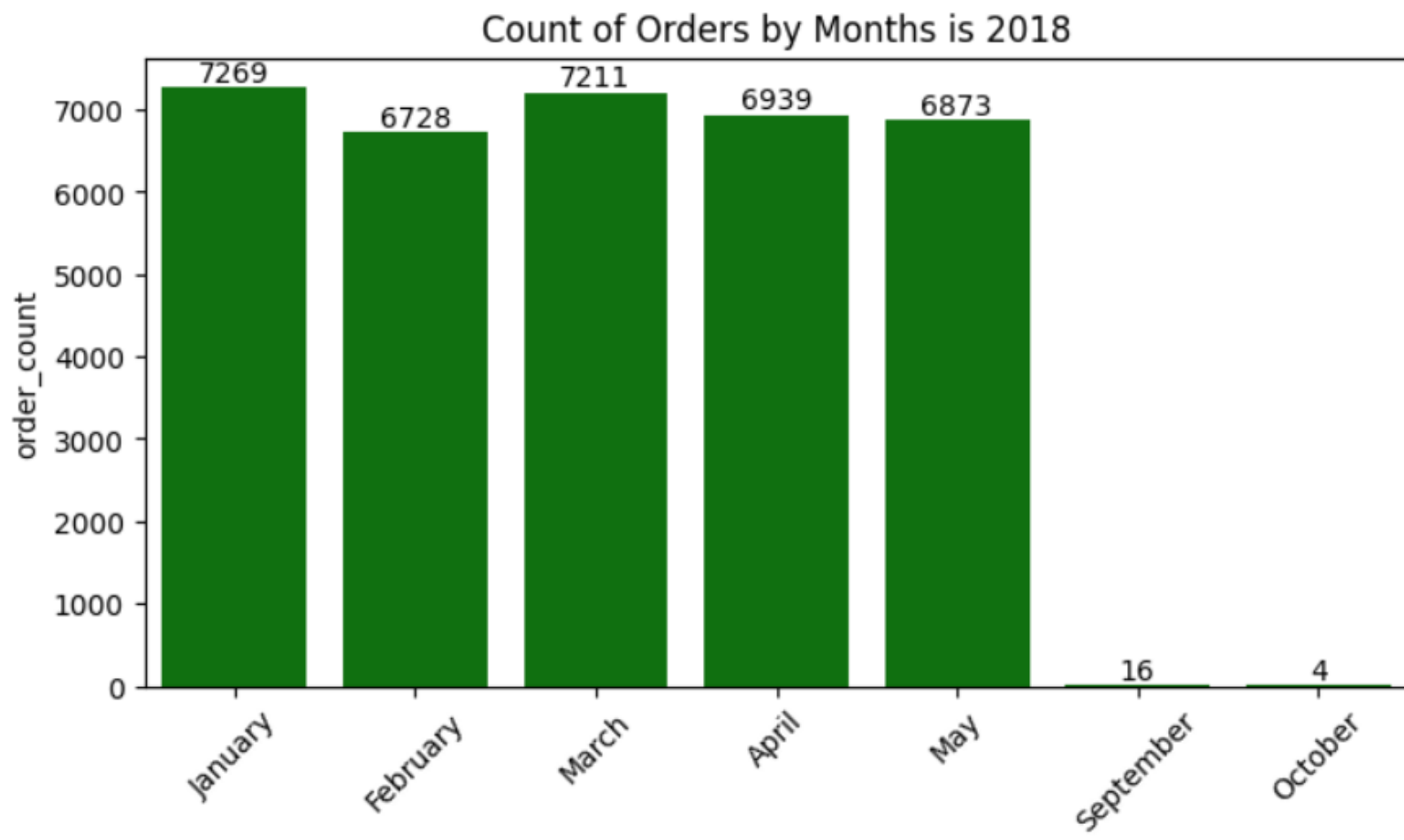
Calculate the number of orders per month in 2018

```
[32]: query = """select monthname(order_purchase_timestamp) months,count(order_id) order_count
from orders where year(order_purchase_timestamp) = 2018
group by months
"""
```



```
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data, columns = ["months", "order_count"])
plt.figure(figsize = (8,4))
o = ["January", "February", "March", "April", "May", "September", "October"]

ax = sns.barplot(x = df["months"], y = df["order_count"], data = df, order = o, color = "green")
plt.xticks(rotation= 45)
ax.bar_label(ax.containers[0])
plt.title("Count of Orders by Months is 2018")
plt.show()
```



Find the average number of products per order, grouped by customer city.

```
[7]: query = """with count_per_order as
(select orders.order_id,orders.customer_id,count(order_items.order_id) as oc
from orders join order_items
on orders.order_id = order_items.order_id
group by orders.order_id,orders.customer_id)

select customers.customer_city,round(avg(count_per_order.oc),2) average_orders
from customers join count_per_order
on customers.customer_id = count_per_order.customer_id
group by customers.customer_city order by average_orders
"""

cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data,columns=["customer city","average orders"])
df.head(10)
```

```
[7]:
```

	customer city	average orders
0	sonora	1.00
1	porteiras	1.00
2	itabi	1.00
3	silvanopolis	1.00
4	salmourao	1.00

5	cha de alegria	1.00
6	cerro grande	1.00
7	francisco santos	1.00
8	barras	1.00
9	grupiara	1.00

Calculate the percentage of total revenue contributed by each product category.

```
[12]: query = """select upper(products.product_category) category,
round((sum(payments.payment_value)/(select sum(payment_value) from payments))*100,2) sales_percentage
from products join order_items
on products.product_id = order_items.product_id
join payments
on payments.order_id = order_items.order_id
group by category order by sales_percentage desc"""

cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data,columns=["Category","percentage distribution"])
df.head(10)
```

[12]:

	Category	percentage distribution
0	BED TABLE BATH	10.70
1	HEALTH BEAUTY	10.35
2	COMPUTER ACCESSORIES	9.90
3	FURNITURE DECORATION	8.93
4	WATCHES PRESENT	8.93
5	SPORT LEISURE	8.70
6	HOUSEWARES	6.84
7	AUTOMOTIVE	5.32
8	GARDEN TOOLS	5.24
9	COOL STUFF	4.87

- ▼ Identify the correlation between product price and the number of times a product has been purchased ¶

[8]:

```
query = """select products.product_category,
count(order_items.product_id),
round(avg(order_items.price),2)
from products join order_items
on products.product_id = order_items.product_id
group by products.product_category;"""
```

```

cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data,columns=["Category","order_count","price"])
df.head(10)

arr1 = df["order_count"]
arr2 = df["price"]

a = np.corrcoef([arr1,arr2])
print("the correlation between price and number of times a product has been purchased is",a[0][1])

```

the correlation between price and number of times a product has been purchased is -0.10631514167157562

Calculate the total revenue generated by each seller, and rank them by revenue

```

query = """select *, dense_rank() over(order by revenue desc) as rn from
(select order_items.seller_id,sum(payments.payment_value)
revenue from order_items join payments
on order_items.order_id = payments.order_id
group by order_items.seller_id) as a """

cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data,columns=["seller_id","revenue","rank"])
df = df.head(10)
sns.barplot(x = "seller_id",y = "revenue", data = df)
ax = sns.barplot(x = df["seller_id"],y =df["revenue"], data = df,color = "green")
plt.xticks(rotation = 45)
ax.bar_label(ax.containers[0])
plt.title("Total revenue by each sellers")
plt.show()

```

Total revenue by each sellers

